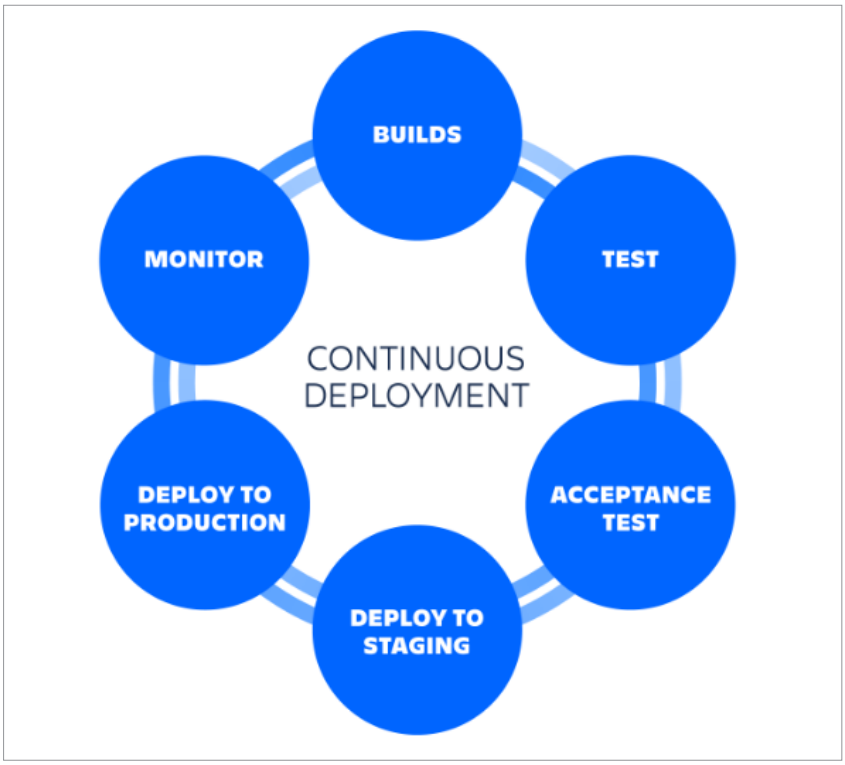

Figure 1: Continuous deployment

making applications highly portable and scalable.

This is where Docker Hub plays a vital role. Docker Hub is a cloud-based registry service provided by Docker Inc., serving as a central hub for storing and sharing container images. Just like GitHub is to code, Docker Hub is to Docker Images. Developers and teams can push their custom images to Docker Hub and pull ready-made images from it, enabling seamless collaboration and reuse. Docker Hub supports both public repositories—which are open to the community—and private repositories**,** allowing teams to manage proprietary images securely.

Using Docker Hub within a CD pipeline offers several key benefits. First, it enables version control for images, allowing developers to tag and track different builds of their applications. This makes it easier to roll back to a previous version in case of bugs or regressions. Second, it provides a centralised, always-available storage solution, ensuring that the latest builds are accessible from any environment, whether it's a developer's local machine or a production cluster. Third, it facilitates easy image pulling, meaning any server or CI/CD agent can quickly retrieve the latest version of an application image to deploy or test. Most importantly, Docker Hub acts as the artifact repository for modern DevOps workflows: CI/CD pipelines can build new images from source code, push them to Docker Hub, and then pull them for deployment—closing the loop for fully automated, reliable continuous deployment.

GitHub Actions: Automation for software workflows: In the realm of continuous deployment, automation is the heartbeat of a streamlined and error-free delivery pipeline. This is where GitHub Actions comes into play—a powerful workflow automation tool built right into GitHub that enables developers to define and run CI/CD pipelines directly within their repositories. GitHub Actions allows you to write workflows using YAML configuration files, where each workflow is made up of one or more jobs that run a sequence of steps, including commands, scripts, or third-party actions. These workflows can be triggered automatically based on a variety of events—such as a push to the repository, a pull request, or even a scheduled cron job—making it a flexible solution for continuous integration, testing, building, and deployment.

With GitHub Actions, developers can automate the entire software lifecycle. For example, when code is pushed to the repository, a workflow can automatically run tests, build a Docker image, push it to Docker Hub, and then deploy it to a cloud server—all without any manual intervention. This aligns perfectly with the principles of continuous deployment, where the goal is to minimise human involvement and maximise reliability and speed. GitHub Actions also provides a marketplace of reusable actions, such as pre-built steps for logging into Docker Hub, setting up programming environments, or deploying to services like AWS, Azure, and Kubernetes—making complex workflows much simpler to implement.

The tight integration between GitHub Actions and GitHub repositories enhances transparency and traceability. Every workflow run is logged and can be inspected directly from the GitHub interface, making debugging and auditing easy. Teams can collaborate better by reviewing workflow outcomes as part of the pull request process. Moreover, GitHub Actions supports matrix builds, allowing simultaneous testing across multiple versions of dependencies or environments, improving software robustness. For teams adopting continuous deployment, GitHub Actions serves as the automation engine that connects every step—from commit to deployment—ensuring that only thoroughly tested, production-ready code makes it into the hands of users.